

Fuzz Testing Report

Revision

Version 9
4/28/26 8:28 PM

SME

Process: Charles Wilson
Report: Marwan Abi-Antoun

Abstract

This document describes the process to create a fuzz testing report.

Group / Owner

Security / Vulnerability Assessment Analyst

Motivation

This document is motivated by the need to have runtime-specific, security-related feedback in the development of software for use within safety-critical, cyber-physical systems.

License

This work was originally created by **Motional** and is licensed under the **Creative Commons Attribution-Share Alike (CC BY-SA-4.0)** License.

<https://creativecommons.org/licenses/by/4.0/legalcode>

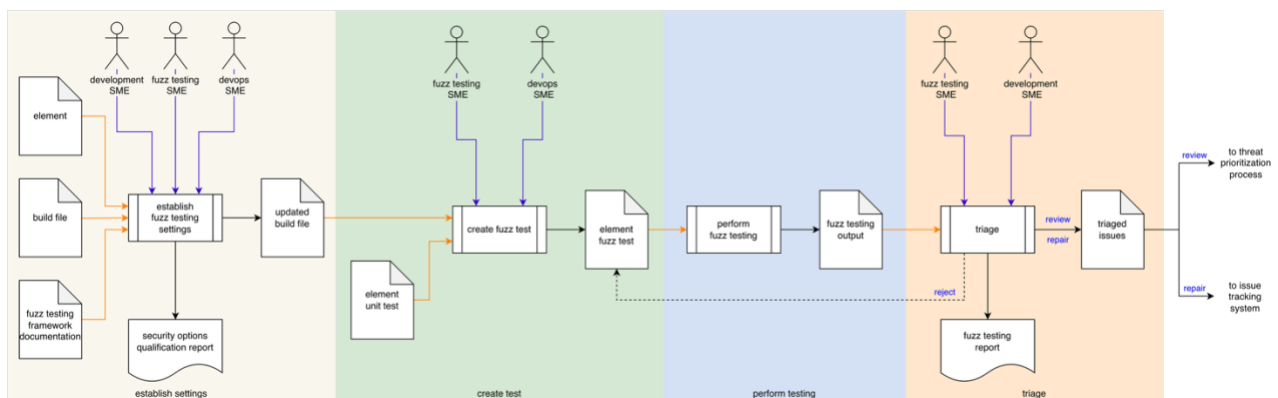
Overview

Although cybersecurity-relevant feedback from tooling such as compilers, static and dynamic analyzers greatly aids in the creation of secure software, there remain situations which these tools do not consider. Fuzz testing [2] provides coverage for some of those situations.

Although lacking the level of automation of other methods, fuzz testing allows for highly tailored behavior analysis at the unit level.

Note: Given the nature of this type of analysis (using invalid, unexpected, or random data as inputs), it is important to not treat fuzz testing as a replacement for more traditional unit testing [7].

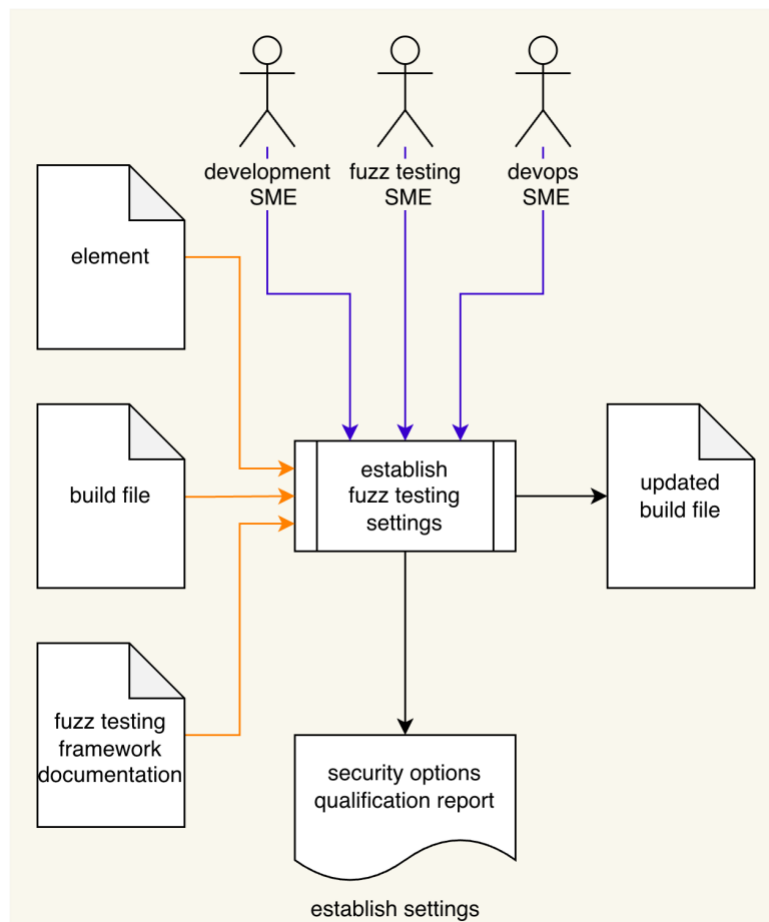
The following diagram illustrates the process to be used:



Process

Establish Settings

Inputs	Build file Element Fuzz testing framework documentation
Outputs	Updated build file Security options qualification report
Participants	Development SME Fuzz Testing SME DevOps SME

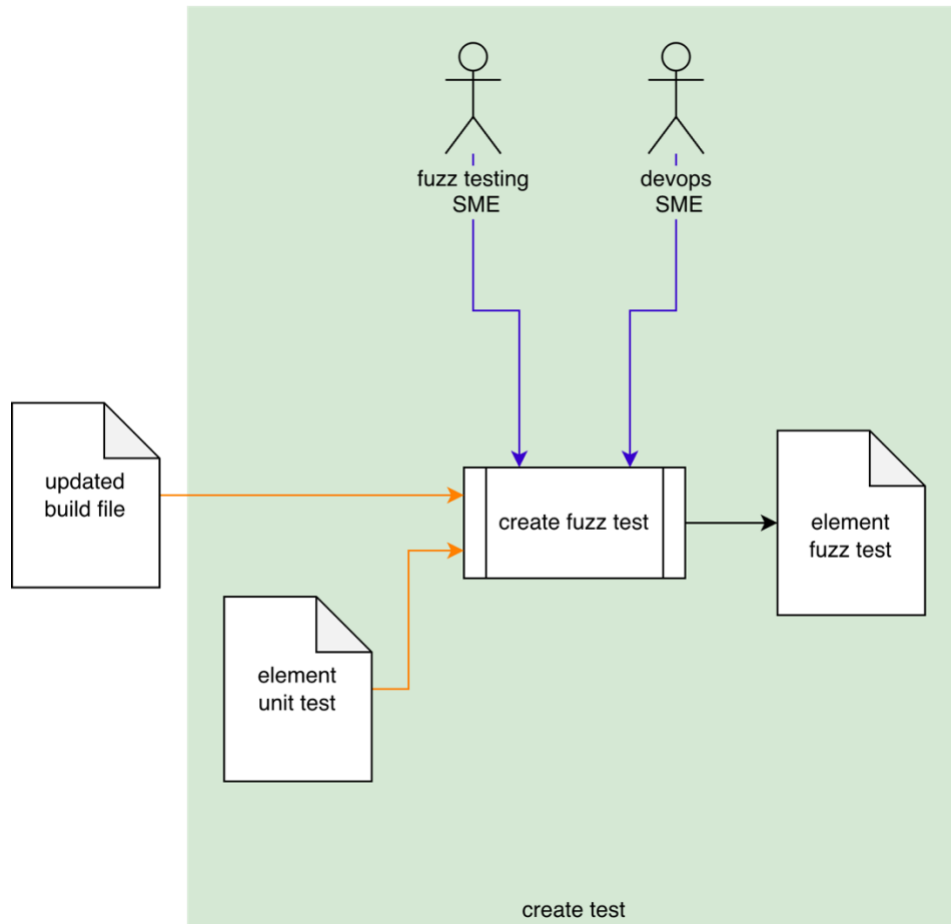


Using the **Element** under consideration, **Fuzz Testing Framework Documentation**, and **Build File**; the **Development SME**, **Fuzz Testing SME**, and **DevOps SME** will work together following the **Secure Settings** ^[4] process to determine what element aspect needs to be fuzz tested and whether the element will require any kind of instrumentation to enable that testing. An **Updated Build File** will be produced. A **Security Options Qualifications Report** is generated.

Note: The scope of the **Element** may be as small as a single file or as large as the entire project.

Create Test

Inputs	Updated build file Element unit test
Outputs	Element fuzz test
Participants	Fuzz Testing SME DevOps SME



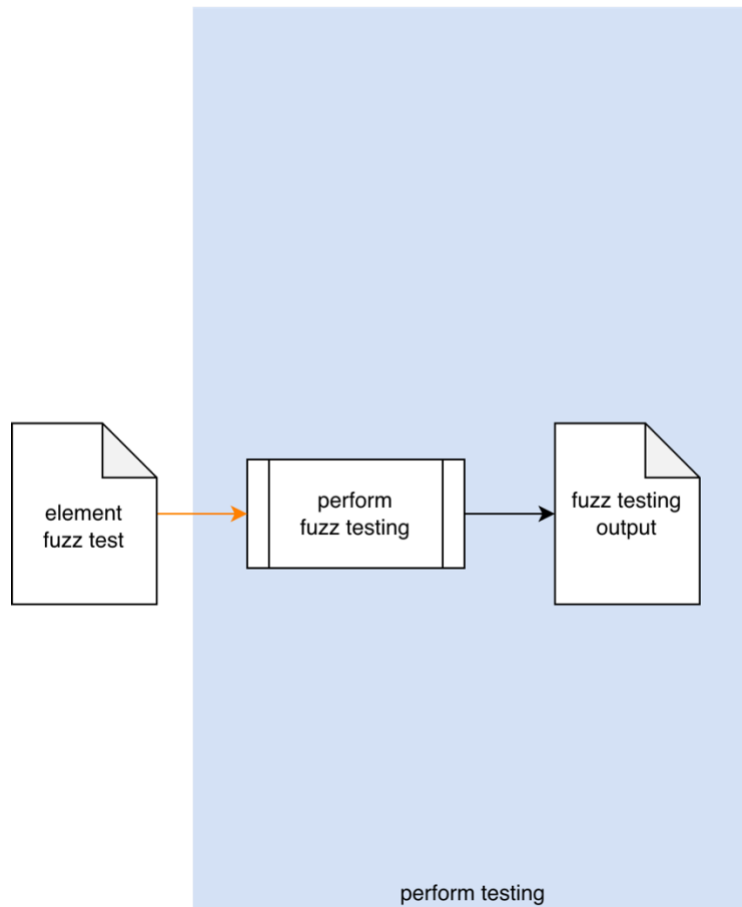
Using the **Updated Build File**, the **Fuzz Testing SME** and **DevOps SME** take the **Element Unit Test** and create an **Element Fuzz Test**.

Note: This activity may be done without the assistance of the **DevOps SME**. It may also be fully automated.

Note: It is not recommended to fuzz test elements which have no existing element unit tests.

Perform Testing

Inputs	Element fuzz test
Outputs	Fuzz testing output
Participants	None

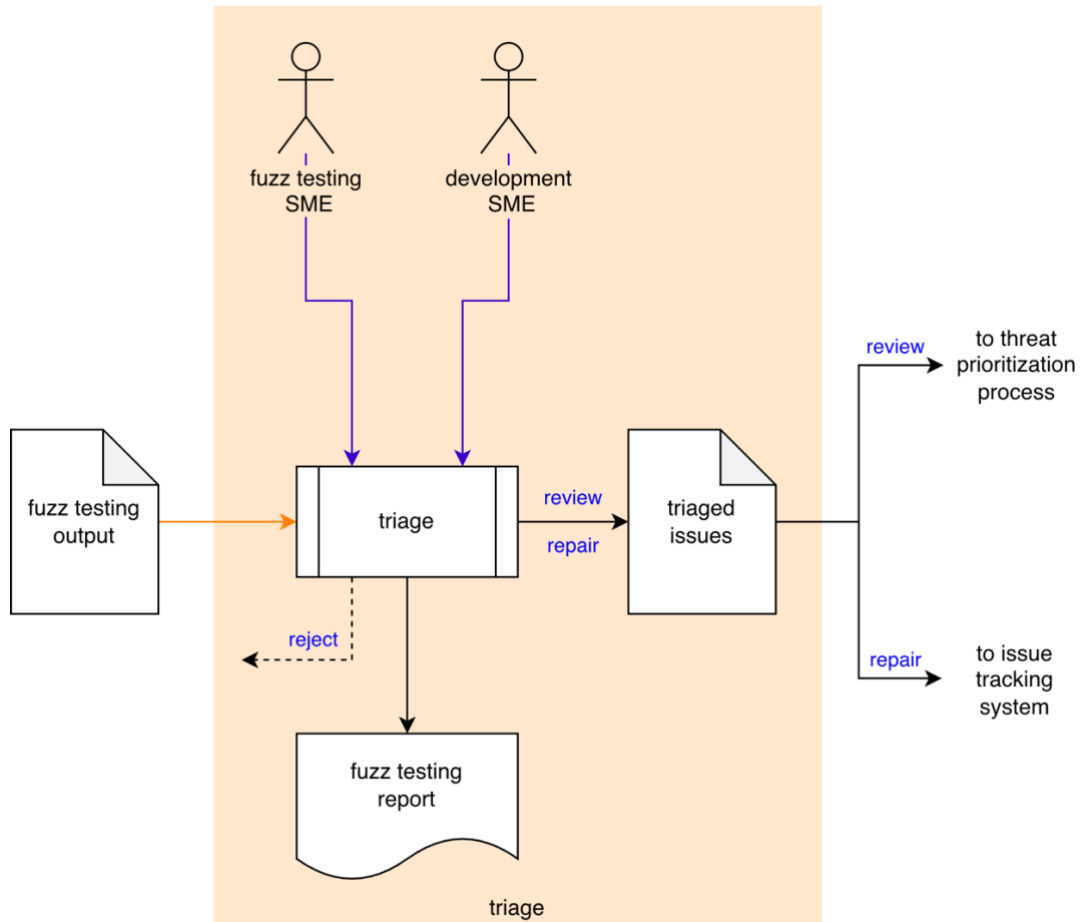


The build invokes the **Element Fuzz Test** which perform fuzz testing on the element under consideration. The results in the generation of **Fuzz Testing Output**.

Note: This activity may be performed manually.

Triage

Inputs	Fuzz testing output
Outputs	Identified issues Fuzz testing Report
Participants	Fuzz Testing SME Development SME



The **Fuzz Testing SME** and **Development SME** review the **Fuzz Testing Output** to identify any anomalous behavior needing further investigation. A **Triaged Issues** list is created, and a **Fuzz Testing Report** is generated.

Issues requiring further review are passed along to the **Threat Prioritization Process** [3]. Issues determined to require immediate attention are entered directly into the **Issue Tracking System**.

Rejected issues should be addressed by updating the **Element Fuzz Test**.

Triaged Issues

The recommended form of the **triaged Issues** artifact is a SARIF-encoded JSON. This document assumes SARIF version 2.1.0 [\[5\]](#) or later.

Fuzz Testing Report

The human-readable **Fuzz Testing Report** is recommended to be produced from the **triaged Issues** artifact and should detail the exposed issues.

The report contains one or more analysis runs. Each run includes:

- Description of the tool used
- Description of the analyzed element
- Tool invocation settings
- Results of the analysis

The tool description includes:

- Name
- Version
- URI to tool documentation
- Tool rules

The tool rules (one or more) convey the classes of analysis performed. Each includes:

- ID (unique)
- Name
- Short description of the rule
- Full description of the rule
- URI to rule documentation
- Reference (optional) to associated taxonomy entry (Common Weakness Enumeration [\[1\]](#), ...)

The analyzed element description (one or more) provides information related to the element under consideration. Each includes:

- URI to analyzed element
- URI to repository the element came from

The analysis results (one or more) describe the issues exposed by the analysis. Each includes:

- Human-readable description
- Location within the element of the issue
- Severity of the issue
- Reference (optional) to the associated taxonomy entry

Note: The above information is directly provided for by the SARIF specification.

It is recommended to include the following additional fuzz testing-specific information:

- Type of exposure (first-order attack surface)
- Type of external data (sensor, socket, file, ...)
- Type of parsing performed
- Triage level (regular, additional dynamic analysis, specific sanitizers, ...)
- Fuzzing harness compiler (`gcc`, `gcc-afl`, `clang-libfuzzer`, ...)
- Type of harness (standalone, library, dual [recommended])

Note: This set of information is not provided for by the baseline SARIF specification but is supported by the SARIF specification's extension capability.

Note: When extending the SARIF encoding to include the above, a namespace to denote fuzz testing (`Fuzzing::tag`) is recommended.

Note: Details on the generation of reports is covered in the **Report Generation** ^[6] AVCDL elaboration document.

References

1. **Common Weakness Enumeration (CWE)**
<https://cwe.mitre.org/>
2. **Fuzzing**
<https://en.wikipedia.org/wiki/Fuzzing>
3. **Threat Prioritization Process** (AVCDL secondary document)
4. **Secure Settings Document** (AVCDL secondary document)
5. **Static Analysis Results Interchange Format (SARIF) Version 2.1.0**
<https://docs.oasis-open.org/sarif/sarif/v2.1.0/os/sarif-v2.1.0-os.pdf>
6. **Report Generation** (AVCDL elaboration document)
7. **Unit testing**
https://en.wikipedia.org/wiki/Unit_testing
8. **Building Secure and Reliable Systems, Chapter 13 – Testing Code**
<https://google.github.io/building-secure-and-reliable-systems/raw/ch13.html>
9. **Trail of Bits: Testing Handbook – Fuzzing**
<https://appsec.guide/docs/fuzzing/>